

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA



DOCUMENTO DE DISEÑO DE SOFTWARE (SDD) SISTEMA DE SOLICITUDES

Diego Iván Correa Navarrete
Juan Antonio González del Río
Diego Arturo Ramos Ávila
Ángel David Carlos Silva

LICENCIATURA EN INGENIERÍA DE SOFTWARE

Junio de 2026

Índice general

1. Introducción	2
1.1. Propósito	2
1.2. Alcance	2
1.3. Definiciones, acrónimos y abreviaturas	2
1.4. Referencias	3
1.5. Panorama del documento	3
2. Arquitectura del sistema	4
2.1. Estilo arquitectónico	4
2.2. Flujo de una petición	5
2.3. Manejo de errores	6
2.4. Reglas arquitectónicas transversales	6
2.5. Organización del proyecto	7
3. Descomposición en módulos	8
3.1. <code>_shared</code> — infraestructura compartida	8
3.2. <code>usuarios</code> — identidad, roles y perfil	8
3.3. <code>solicitudes</code> — núcleo	9
3.3.1. <code>tipos</code> — catálogo de tipos de solicitud	9
3.3.2. <code>formularios</code> — constructor de formularios en tiempo de ejecución	9
3.3.3. <code>intake</code> — alta de solicitudes (solicitante)	9
3.3.4. <code>revision</code> — atención de solicitudes (personal)	9
3.3.5. <code>lifecycle</code> — ciclo de vida y folios	9
3.3.6. <code>archivos</code> — adjuntos del solicitante	10
3.3.7. <code>pdf</code> — plantillas y generación de PDF	10
3.3.8. <code>plantilla_assets</code> — biblioteca de imágenes	10
3.3.9. <code>respuesta</code> — entrega de respuesta (personal)	10
3.4. <code>mentores</code> — catálogo de mentores	10
3.5. <code>notificaciones</code> — despacho de correo	10
3.6. <code>reportes</code> — tablero y exportaciones	11
4. Modelo de datos	12
4.1. Diagrama entidad-relación	12
4.2. Entidades clave	13
4.3. Integridad y consistencia	15
5. Decisiones de diseño y patrones	16
5.1. DTOs de Pydantic en las fronteras	16
5.2. Inyección de dependencias por funcionalidad	16
5.3. Máquina de estados del ciclo de vida	16

5.4.	Folios únicos	17
5.5.	Generación de PDF con WeasyPrint	17
5.6.	Almacenamiento de archivos transaccional	17
5.7.	Auditoría	17
6.	Diseño de interfaz de usuario	18
6.1.	Distribución general	18
6.2.	Componentes compartidos	18
6.3.	Vistas por rol	18
6.4.	Accesibilidad y diseño responsivo	19
7.	Seguridad	20
7.1.	Autenticación delegada (JWT)	20
7.2.	Autorización en capas	20
7.3.	CSP en la previsualización de plantillas	20
7.4.	Validación de archivos	21
7.5.	Transporte y entornos	21
8.	Tecnologías y dependencias	22
8.1.	Integraciones externas	23
8.2.	Convenciones de construcción	23

Capítulo 1

Introducción

1.1. Propósito

Este documento describe el diseño del software (*Software Design Description*, SDD) del **Sistema de Solicitudes** de la Universidad Autónoma de Zacatecas (UAZ). Su objetivo es traducir los requerimientos funcionales y no funcionales descritos en la Especificación de Requerimientos de Software (SRS) a una arquitectura concreta, una descomposición en módulos, un modelo de datos y un conjunto de decisiones de diseño que guíen la construcción, prueba y mantenimiento del sistema.

El SDD está dirigido a las personas que implementan, prueban, revisan o mantienen el sistema: equipo de desarrollo, responsables de aseguramiento de calidad y personal de operación. Presupone familiaridad con Python, Django y los conceptos de arquitectura por capas.

1.2. Alcance

El Sistema de Solicitudes es una aplicación web monolítica que digitaliza la gestión de solicitudes académicas y administrativas de la UAZ, reemplazando procesos manuales basados en papel y correo por un flujo trazable. El sistema permite que estudiantes y docentes presenten solicitudes mediante formularios dinámicos configurables, da seguimiento al ciclo de vida de cada solicitud mediante una máquina de estados, genera documentos PDF a partir de plantillas, notifica por correo a las partes involucradas y ofrece administración del catálogo, gestión de archivos adjuntos, atención por roles y reportes agregados.

El sistema delega la autenticación a un proveedor externo (recibe un token JWT) y consulta datos de usuario al sistema institucional SIGA. No procesa pagos: únicamente recibe comprobantes como archivos adjuntos.

1.3. Definiciones, acrónimos y abreviaturas

Término	Significado
SDD	<i>Software Design Description</i> : documento de diseño del software.
SRS	<i>Software Requirements Specification</i> : especificación de requerimientos.

Término	Significado
DTO	<i>Data Transfer Object</i> : objeto inmutable que transporta datos entre capas.
ORM	<i>Object-Relational Mapping</i> : capa de mapeo objeto-relacional de Django.
ABC	<i>Abstract Base Class</i> : clase base abstracta (interfaz) de Python.
DI	<i>Dependency Injection</i> : inyección de dependencias.
JWT	<i>JSON Web Token</i> : token firmado emitido por el proveedor de autenticación.
SIGA	Sistema de Información para la Gestión Académica de la UAZ.
CSP	<i>Content Security Policy</i> : política de seguridad de contenido HTTP.
Folio	Identificador único de una solicitud con formato SOL-AAAA-NNNNN.
Plantilla	Documento HTML+CSS administrable que se renderiza a PDF.
Snapshot	Fotografía inmutable de la definición de campos al crear una solicitud.
RF / RNF	Requerimiento funcional / requerimiento no funcional.

1.4. Referencias

- Especificación de Requerimientos de Software (SRS) del Sistema de Solicitudes.
- `specs/global/architecture.md` — arquitectura global (fuente principal de este diseño).
- `specs/global/requirements.md` — requerimientos funcionales (RF) y no funcionales (RNF).
- `specs/apps/*/design.md` — diseño detallado por funcionalidad.
- `.claude/rules/django-code-architect.md` — reglas arquitectónicas canónicas.
- Diagrama entidad-relación: `entregables/documentos/esquema_bd_diagrama.pdf`.

1.5. Panorama del documento

El Capítulo 2 describe la arquitectura del sistema y el flujo de una petición. El Capítulo 3 descompone el sistema en aplicaciones y funcionalidades. El Capítulo 4 presenta el modelo de datos y el diagrama entidad-relación. El Capítulo 5 documenta las decisiones de diseño y los patrones aplicados. Los Capítulos 6, 7 y 8 cubren el diseño de interfaz de usuario, la seguridad y las tecnologías y dependencias, respectivamente.

Capítulo 2

Arquitectura del sistema

2.1. Estilo arquitectónico

El sistema sigue un estilo **Vista** $\rightarrow$ **Servicio** $\rightarrow$ **Repositorio** (*View* $\rightarrow$ *Service* $\rightarrow$ *Repository*), con objetos de transferencia de datos (DTO) tipados con **Pydantic v2** en cada frontera entre capas. Es una aplicación **monolítica** con **plantillas renderizadas en el servidor** (Django Templates con Tailwind CSS v4 y Alpine.js v3); deliberadamente **no** expone una API REST con DRF.

Cada capa tiene una responsabilidad única y bien delimitada:

- **Vista (Vista, frontera HTTP).** Posee las preocupaciones HTTP: recibe la petición, resuelve el actor a partir del JWT, instancia el formulario, traduce `cleaned_data` a un DTO de entrada, invoca el servicio y renderiza la plantilla o emite la redirección. Nunca llama directamente a un repositorio ni al ORM.
- **Formulario.** Analiza y valida la entrada del usuario y convierte `cleaned_data` en un DTO de Pydantic antes de cruzar hacia el servicio.
- **Servicio (lógica de negocio).** Python puro, definido como una clase base abstracta (ABC) más su implementación. Contiene la lógica de dominio, la autorización por política (p. ej. “solo el dueño edita”) y la orquestación transaccional. No conoce HTTP ni el ORM.
- **Repositorio (acceso a datos).** Único lugar donde vive el ORM de Django. Definido como ABC más implementación. Recibe y devuelve DTOs; traduce toda excepción de Django (`Model.DoesNotExist`) a una excepción de dominio antes de que escape de la capa.

Tabla 2.1: Capas y su responsabilidad (de la frontera HTTP hacia la base de datos).

Capa	Responsabilidad	Conoce
Vista	HTTP, autorización por rol (mixins), renderizado de plantillas.	Formularios, Servicios
Formulario	Parseo y validación de entrada; producción de DTO.	Schemas (DTO)
Servicio	Lógica de negocio, autorización por política, transacciones.	Repositorios, otros Servicios (vía interfaz)
Repositorio	Consultas y escrituras ORM; traducción a DTO y a excepciones.	ORM, Modelos
ORM / BD	Persistencia en PostgreSQL (SQLite en pruebas internas).	—

2.2. Flujo de una petición

El recorrido de una petición autenticada atraviesa las capas de forma descendente y regresa por la misma ruta con DTOs:

1. La petición HTTP entra por el `RequestIDMiddleware`, que asigna un identificador de correlación, y por el `JwtAuthenticationMiddleware`, que valida el token (cookie `stk` o cabecera `Authorization: Bearer`), fija `request.user` y `request.user_dto`.
2. La **Vista** aplica los *mixins* de permiso (`LoginRequiredMixin`, mixins por rol), construye el formulario con `request.POST/request.FILES` y lo valida.
3. El **Formulario** produce un **DTO de entrada** de Pydantic a partir de `cleaned_data`.
4. La Vista invoca al **Servicio** pasándole el DTO y el actor (`UserDTO`).
5. El **Servicio** ejecuta la lógica de dominio y delega la persistencia al **Repositorio** dentro de `transaction.atomic()` cuando aplica.
6. El **Repositorio** ejecuta el ORM y devuelve un **DTO de salida**.
7. El Servicio devuelve el DTO a la Vista, que lo entrega a la plantilla para su renderizado en el servidor.

Tabla 2.2: Esquema textual del flujo de una petición.

HTTP Request
↓ RequestIDMiddleware → JwtAuthenticationMiddleware
Vista (frontera HTTP) — Formulario (parsea + valida) → DTO
↓
Servicio (lógica de negocio, ABC + impl.)
↓
Repositorio (solo ORM, ABC + impl.) → DTO
↓
Django ORM / PostgreSQL

2.3. Manejo de errores

Todas las excepciones de dominio heredan de `_shared.exceptions.AppError`, que centraliza los *centinelas* reutilizables: `NotFound` (404), `Conflict` (409), `Unauthorized` (403), `AuthenticationRequired` (401), `DomainValidationError` (422) y `ExternalServiceError` (502). Cada excepción concreta de una funcionalidad es subclase de uno de estos centinelas y declara su `code` y su `user_message` en español.

El `AppErrorMiddleware` actúa como red de seguridad: en `process_exception` captura cualquier `AppError`, mapea el estado HTTP correspondiente, registra el evento junto con el `request_id` y renderiza `_shared/error.html` (o devuelve JSON para peticiones AJAX). Las vistas pueden capturar `AppError` selectivamente para devolver `field_errors` al formulario y re-renderizarlo con las marcas de error.

Tabla 2.3: Centinelas de `AppError` y su estado HTTP.

Centinela	HTTP	Uso
<code>AuthenticationRequired</code>	401	Token ausente, inválido o expirado.
<code>Unauthorized</code>	403	Rol o política de dominio no autorizada.
<code>NotFound</code>	404	Recurso inexistente.
<code>Conflict</code>	409	Transición de estado inválida, conflicto de unicidad.
<code>DomainValidationError</code>	422	Validación de negocio fallida (con <code>field_errors</code>).
<code>ExternalServiceError</code>	502	SIGA o SMTP no responde.

2.4. Reglas arquitectónicas transversales

- **El ORM nunca escapa del repositorio.** Solo el repositorio importa modelos y ejecuta `Model.objects`.

- **Dependencia entre funcionalidades.** Un servicio solo accede a los repositorios de su propia funcionalidad. Para leer datos de otra funcionalidad inyecta la *interfaz de servicio* de esa funcionalidad, nunca su repositorio. Cuando el consumidor define la necesidad, declara un *puerto* (ABC) y el productor provee el *adaptador*.
- **DTOs en cada frontera.** Pydantic v2 para DTOs; nunca diccionarios desnudos cruzando capas.
- **Una clase pública por archivo.** Modelos, servicios y repositorios se definen uno por archivo.
- **Identificadores en inglés; texto al usuario en español.**

2.5. Organización del proyecto

El repositorio Git contiene en su raíz únicamente la infraestructura (`Dockerfile`, `docker-compose.*.ym`, `Makefile`, `.env.example`). Todo el código Django vive dentro de `app/`, montado en `/app/` dentro del contenedor. Cada aplicación de Django es un “corte vertical” por capas; dentro de cada aplicación, cada funcionalidad (*feature*) es su propio paquete con esta estructura típica:

Tabla 2.4: Estructura de paquete de una funcionalidad.

Archivo / carpeta	Contenido
<code>schemas.py</code>	DTOs de Pydantic.
<code>exceptions.py</code>	Excepciones específicas (subclases de <code>_shared</code>).
<code>repositories/<x>/</code>	<code>interface.py</code> (ABC) + <code>implementation.py</code> (ORM).
<code>services/<x>/</code>	<code>interface.py</code> (ABC) + <code>implementation.py</code> .
<code>views/<actor>.py</code>	Vistas por actor (solicitante, personal, admin).
<code>forms/</code>	Un formulario por archivo.
<code>dependencies.py</code>	Funciones de fábrica para inyección de dependencias.
<code>permissions.py</code>	Mixins y decoradores de autorización.
<code>constants.py</code> , <code>urls.py</code> , <code>tests/</code>	Constantes, ruteo y pruebas por capa.

Los modelos ORM viven a nivel de *aplicación* (`<app>/models/`), uno por archivo, compartidos por todas las funcionalidades de esa aplicación.

Capítulo 3

Descomposición en módulos

El sistema se descompone en seis aplicaciones de Django más un paquete de infraestructura compartida. La aplicación `solicitudes` es el núcleo y se subdivide en varias funcionalidades.

3.1. `_shared` — infraestructura compartida

Contiene infraestructura transversal, **sin lógica de dominio**: la jerarquía de excepciones (`AppError` y centinelas), los *middlewares* (`RequestIDMiddleware`, `StructuredLoggingMiddleware`, `AppErrorMiddleware`), los ayudantes de validación JWT (`auth.py`, Python puro sin dependencia de `HttpRequest`), la paginación (`PageRequest` y `Page[T]`), el ayudante de auditoría (`audit.py`) y el envoltorio de `WeasyPrint` (`pdf.py`).

3.2. `usuarios` — identidad, roles y perfil

Responsabilidad. Validación del JWT en *middleware*, modelo de usuario propio (extensión de `AbstractBaseUser`, llave primaria `matricula`), catálogo de roles y *mixins* de permiso. No hay vistas de *login* o registro: la autenticación es externa.

Capas y piezas principales.

- **DTOs:** `UserDT0` (incluye `is_mentor`, poblado bajo demanda por mentores), `CreateOrUpdateUserInput` y `SigaProfile`.
- **Servicios:** `UserService` (`get_or_create_from_claims`, `get_by_matricula`, `hydrate_from_siga`, `list_by_role`), `RoleResolver` (claim del proveedor $\rightarrow$ `Role`) y `SigaService` (consulta HTTP a SIGA con *timeout*; mejor esfuerzo).
- **Roles** (`Role`): `ALUMNO`, `DOCENTE`, `CONTROL_ESCOLAR`, `RESPONSABLE_PROGRAMA`, `ADMIN`.
- **Middleware** `JwtAuthenticationMiddleware`: decodifica el JWT, fija `request.user` y `request.user_dto`, redirige al *login* externo si el token es inválido.
- **Mixins** de permiso: `LoginRequiredMixin`, `RoleRequiredMixin` y los presets `Alumno/Docente/Persona`.

Subfunción `directory`. Directorio de usuarios de solo lectura para administradores (`UserDirectoryService`, listado paginado con filtros y detalle); superpone `is_mentor` consultando `MentorService`.

3.3. solicitudes — núcleo

Reúne la mayor parte de la lógica de dominio en funcionalidades acopladas débilmente.

3.3.1. tipos — catálogo de tipos de solicitud

El administrador crea y edita tipos sin tocar código. Cada `TipoSolicitud` define nombre, descripción, `responsible_role` (rol que atiende), `creator_roles` (subconjunto de {Alumno, Docente}), `requires_payment`, `mentor_exempt`, plantilla opcional y la definición de campos dinámicos (`FieldDefinition`). Servicios: `TipoService` (`list_for_admin`, `list_for_creator`, `create`, `update`, `deactivate`, `snapshot`). El borrado físico está ausente por diseño: los tipos son *tombstones* para que las solicitudes históricas resuelvan su propio *snapshot*.

3.3.2. formularios — constructor de formularios en tiempo de ejecución

Convierte un `FormSnapshot` en una clase de formulario de Django (`build_django_form`). Mapea cada `FieldType` (TEXT, TEXTAREA, NUMBER, DATE, SELECT, FILE) a su campo y validadores. Excluye los campos de auto-llenado (`source` $\neq$ `USER_INPUT`) como frontera de seguridad. Expone `to_values_dict()` que serializa a primitivos JSON.

3.3.3. intake — alta de solicitudes (solicitante)

El solicitante descubre los tipos que puede presentar, llena el formulario dinámico, adjunta archivos y envía. `IntakeService.create` captura el *snapshot* de campos *al momento del envío*, calcula `pago_exento`, asigna folio y persiste la `Solicitud` y su primer `HistorialEstado` dentro de una transacción atómica; fuera de ella notifica y audita. Posee el puerto `MentorService` (`is_mentor`) y el `AutoFillResolver` para campos auto-llenados desde el perfil SIGA.

3.3.4. revision — atención de solicitudes (personal)

Espejo del lado del personal: cola compartida por rol responsable, vista de detalle con acciones y los tres verbos `atender`, `finalizar` y `cancelar`. `ReviewService` es un envoltorio delgado sobre `LifecycleService` que añade autorización de lectura por rol.

3.3.5. lifecycle — ciclo de vida y folios

Posee los modelos `Solicitud`, `HistorialEstado` y `FolioCounter`, la máquina de estados, la asignación de folios SOL-AAAA-NNNNN, las consultas de lectura y el historial de transiciones de solo anexo. `LifecycleService.transition` valida la transición, autoriza al actor, escribe el nuevo estado y el historial en una transacción y, fuera de ella, notifica y audita. Define los puertos `NotificationService` y el ayudante de auditoría.

3.3.6. archivos — adjuntos del solicitante

Carga y descarga de archivos validados por extensión y tamaño (máx. 10 MB, RT-07). `ArchivoService` valida según el *snapshot*, calcula `sha256`, persiste los bytes mediante una abstracción de almacenamiento (`FileStorage` con renombrado transaccional vía `transaction.on_commit`) y autoriza la descarga (dueño, personal del rol responsable o admin). Semántica de reemplazo al recargar.

3.3.7. pdf — plantillas y generación de PDF

Posee el modelo `PlantillaSolicitud` (HTML+CSS administrable), el CRUD de plantillas y el renderizado bajo demanda de una solicitud a PDF con `WeasyPrint`. No persiste los bytes del PDF: cada descarga re-renderiza desde los datos de origen, con determinismo (mismo `pdf_identifier` + reloj fijo $\Rightarrow$ bytes idénticos). `PdfService` compone `LifecycleService`, `UserService`, `AssetService` y el envoltorio de `WeasyPrint`.

3.3.8. plantilla_assets — biblioteca de imágenes

Imágenes administrables (PNG/JPG/WEBP) referenciadas por las plantillas como `{{ assets.<slug> }}`, con alcance `global` o por `plantilla`. `AssetService.list_for_render` las resuelve a URIs `data:` en tiempo de render. Validación en profundidad (forma, servicio y verificación con PIL).

3.3.9. respuesta — entrega de respuesta (personal)

El personal entrega los documentos resultantes como *lotes* (`RespuestaSolicitud`) de archivos (`ArchivoRespuesta`) con un comentario. `RespuestaService` valida estado (`EN_PROCESO`), rol y archivos, y aplica una matriz de visibilidad: el solicitante ve los lotes solo cuando la solicitud está `FINALIZADA`. Funcionalidad de solo anexo.

3.4. mentores — catálogo de mentores

Mantiene la lista de matrículas registradas como mentores (exentas de comprobante de pago en tipos con `mentor_exempt`). Alta manual o por CSV. La versión historizada (`MentorPeriodo`) preserva cada periodo (`alta`, `baja`) y responde preguntas de pertenencia en un punto del tiempo. `MentorService.is_mentor` es el contrato consumido por `intake` a través de un adaptador.

3.5. notificaciones — despacho de correo

Integración de salida pura, sin modelo ni vista. Implementa el puerto `NotificationService` definido por `lifecycle`. Al crear una solicitud envía correo al rol responsable y acuse al solicitante; en cada cambio de estado notifica al solicitante. Toda falla de SMTP se registra y se absorbe: nunca bloquea la operación de negocio (RF-07).

3.6. reportes — tablero y exportaciones

Tablero administrativo con conteos agregados (por estado, tipo y mes) sobre un conjunto filtrable, listado paginado y exportación a CSV (BOM UTF-8, cursor del lado de la base de datos) y PDF (WeasyPrint, tope de 1000 filas). No añade tablas: lee a través de `LifecycleService` usando agregaciones de una sola consulta SQL.

Capítulo 4

Modelo de datos

4.1. Diagrama entidad-relación

La Figura 4.1 muestra el esquema de la base de datos. Las entidades clave se describen en la sección siguiente.

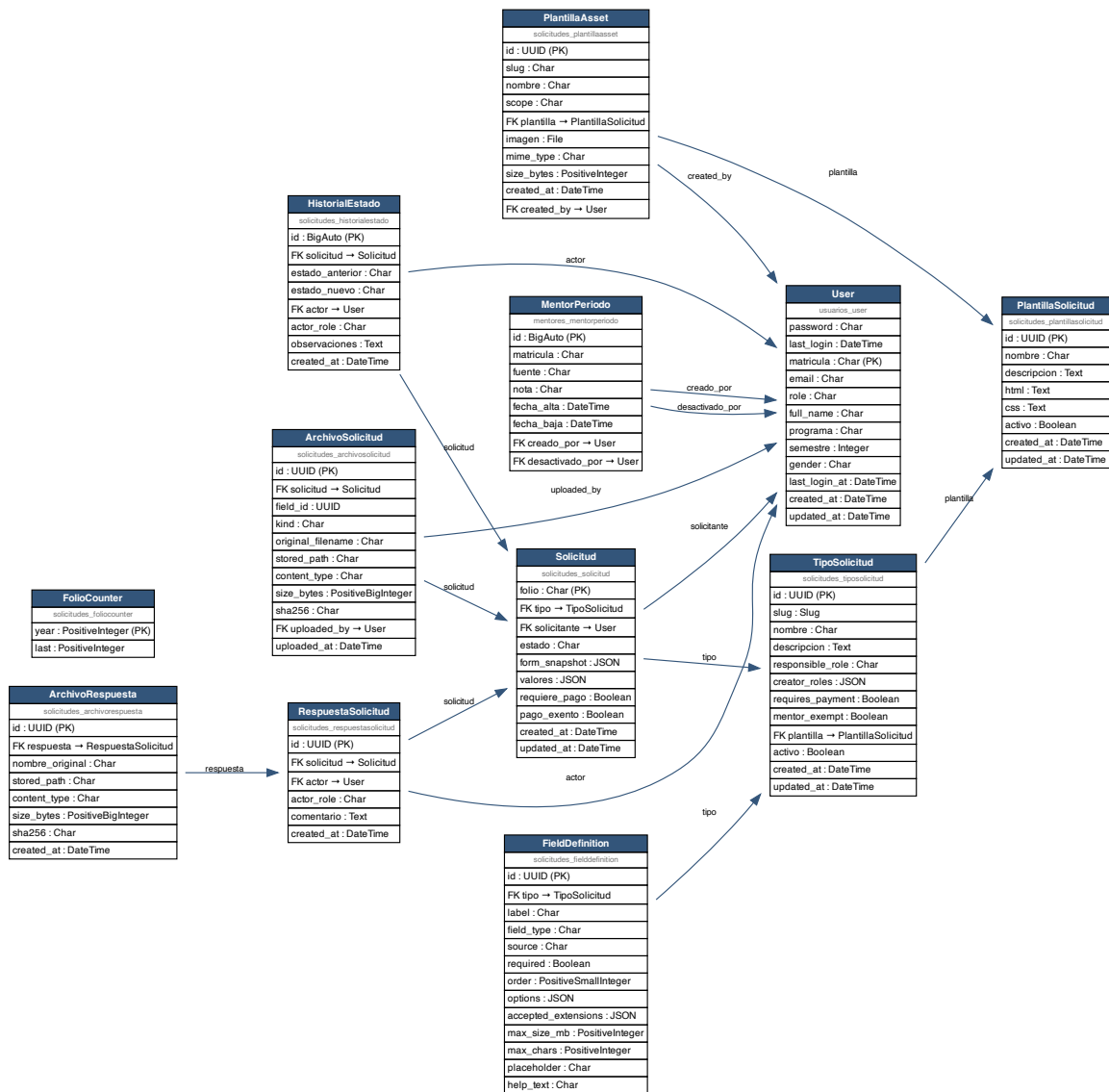


Figura 4.1: Diagrama entidad-relación del Sistema de Solicitudes.

4.2. Entidades clave

User (usuarios)

Llave primaria matricula. Campos: email (único, propiedad del proveedor de auth), role, campos cacheados de SIGA (full_name, programa, semestre, gender), last_login_at y auditoría. Las cadenas vacías nunca sobrescriben valores cacheados (contrato de re-login con solo JWT).

TipoSolicitud (solicitudes)

id (UUID), slug único, nombre, descripcion, responsable_role, creator_roles (JSON),

requires_payment, mentor_exempt, plantilla (FK a PlantillaSolicitud, SET_NULL), activo y auditoría.

FieldDefinition (solicitudes)

id (UUID), tipo (FK CASCADE), label, field_type, required, order (único por tipo), options (SELECT), accepted_extensions / max_size_mb (FILE), max_chars (TEXT/-TEXTAREA), source (auto-llenado).

Solicitud (solicitudes)

Llave primaria folio (SOL-AAAA-NNNNN). Campos: tipo (FK PROTECT), solicitante (FK PROTECT), estado, form_snapshot (JSON, congelado al crear), valores (JSON), requiere_pago, pago_exento y auditoría.

HistorialEstado (solicitudes)

id, solicitud (FK CASCADE), estado_anterior (nulo en el alta), estado_nuevo, actor (FK PROTECT), actor_role (*snapshot*), observaciones, created_at. De solo anexado.

FolioCounter (solicitudes)

year (PK), last. Una fila por año; select_for_update serializa la asignación.

ArchivoSolicitud (solicitudes)

id (UUID), solicitud (FK CASCADE), field_id (UUID, nulo para comprobante), kind (FORM/COMPROBANTE), original_filename, stored_path, content_type, size_bytes, sha256, uploaded_by (FK PROTECT). Restricciones parciales de unicidad por (solicitud, field_id) y por comprobante.

PlantillaSolicitud (solicitudes)

id (UUID), nombre, descripcion, html (cuerpo de plantilla Django), css, activo y auditoría.

PlantillaAsset (solicitudes)

id (UUID), slug, nombre, scope (global/plantilla), plantilla (FK CASCADE, nulo si global), imagen, mime_type, size_bytes, created_by (FK PROTECT). Unicidad de slug por alcance y *check* de consistencia de alcance.

RespuestaSolicitud (solicitudes)

id (UUID), solicitud (FK CASCADE), actor (FK PROTECT), actor_role (*snapshot*), comentario, created_at. Es el *lote* de respuesta.

ArchivoRespuesta (solicitudes)

id (UUID), respuesta (FK CASCADE), nombre_original, stored_path, content_type, size_bytes, sha256, created_at. Un archivo de un lote.

MentorPeriodo (mentores)

id (BigAuto), matricula (indexada), fuente (MANUAL/CSV), nota, fecha_alta, fecha_baja (nulo = periodo abierto), creado_por y desactivado_por (FK PROTECT). Unicidad parcial: un único periodo abierto por matrícula.

4.3. Integridad y consistencia

- **Inmutabilidad del *snapshot*.** `Solicitud.form_snapshot`, `requiere_pago` y `pago_exento` se congelan al crear y no se recalculan al leer, de modo que la auditoría y los reportes son estables aunque el tipo o la lista de mentores cambien después.
- **Borrado protegido.** Las relaciones a `User` y a `TipoSolicitud` usan `PROTECT` para impedir borrados que huérfanen historial o solicitudes.
- **Solo anexado.** `HistorialEstado` y las respuestas no exponen métodos de borrado o actualización en sus repositorios.

Capítulo 5

Decisiones de diseño y patrones

5.1. DTOs de Pydantic en las fronteras

Cada cruce de capa transporta un DTO inmutable (`frozen=True`). Esto desacopla la representación de persistencia (modelos ORM) de la representación de dominio y de presentación, evita que el ORM se filtre a las plantillas y permite validar invariantes en el punto de construcción del DTO mediante validadores de Pydantic (`@model_validator`).

5.2. Inyección de dependencias por funcionalidad

Cada funcionalidad declara su grafo de dependencias en `dependencies.py` mediante *funciones de fábrica* (p.ej. `get_intake_service()`). Las vistas llaman a la fábrica una vez por petición. Esto mantiene a los servicios libres de framework, facilita las pruebas (se inyectan repositorios falsos en memoria) y resuelve ciclos de construcción —por ejemplo, el ciclo entre `LifecycleService` y `NotificationService` se rompe construyendo una instancia de ciclo de vida de solo lectura con un notificador `NoOp` para el notificador.

5.3. Máquina de estados del ciclo de vida

El ciclo de vida de una solicitud se modela como un mapa explícito de transiciones legales:

Tabla 5.1: Transiciones legales de `Solicitud.estado`.

Estado origen	Acción	Estado destino	Quién
CREADA	atender	EN_PROCESO	Personal del rol responsable / admin
EN_PROCESO	finalizar	FINALIZADA	Personal del rol responsable / admin
CREADA	cancelar	CANCELADA	Solicitante (dueño), personal, admin
EN_PROCESO	cancelar	CANCELADA	Personal del rol responsable / admin

Las transiciones prohibidas elevan `InvalidStateTransition` (409). Cada transición exitosa escribe un `HistorialEstado` (actor, rol, fecha, observaciones) y emite una notificación.

Reglas de cancelación: el solicitante solo en `CREADA`; el personal del rol responsable desde `CREADA` o `EN_PROCESO`; el administrador desde cualquier estado no terminal. La cola es *compartida* (no hay campo `assigned_to`); gana la primera escritura ante contención.

5.4. Folios únicos

`FolioCounter` mantiene una fila por año. `OrmFolioRepository.allocate` usa `select_for_update` dentro de una transacción para serializar la asignación e incrementar el contador; `DefaultFolioService.ne` formatea `SOL-{year}-{n:05d}`. El *throughput* queda acotado por la contención de bloqueo a nivel de fila, lo cual es suficiente para el perfil de carga del proyecto.

5.5. Generación de PDF con WeasyPrint

`_shared/pdf.render_pdf` es la única superficie de WeasyPrint del sistema. El `PdfService` compone los datos de la solicitud, el solicitante y la plantilla, construye el contexto de render, evalúa el cuerpo de la plantilla como plantilla de Django y produce los bytes. No se persiste el PDF: cada descarga re-renderiza desde el origen, garantizando consistencia con el estado actual. El `pdf_identifier` combinado con un reloj fijo produce bytes idénticos para la misma entrada (determinismo dentro de un entorno). Las imágenes se incrustan como URIs `data:` para estabilidad entre despliegues.

5.6. Almacenamiento de archivos transaccional

`LocalStorage.save` escribe los bytes en un archivo `.partial` y registra un *hook* `transaction.on_commit` que renombra atómicamente a la ruta final con `os.replace`. Si la transacción externa revierte, el *hook* no se dispara y el `.partial` se limpia con `cleanup_pending()`. Esto evita huérfanos y archivos sin fila. La ruta se resuelve contra `MEDIA_ROOT` rechazando rutas que escapen de la raíz.

5.7. Auditoría

`audit.write(event, **fields)` emite una línea INFO estructurada en el *logger* `audit` con los campos relevantes (sin persistencia en base de datos en la versión 1; el flujo de *logs* JSON es la superficie consultable). Se invoca en `solicitud.creada` y `solicitud.estado_cambiado`, fuera de la transacción, como observabilidad de mejor esfuerzo.

Capítulo 6

Diseño de interfaz de usuario

6.1. Distribución general

La interfaz se renderiza en el servidor con Django Templates, estilizada con Tailwind CSS v4 (enfoque *CSS-first* mediante `@theme`), con interactividad ligera mediante Alpine.js v3 e iconos de Lucide. Existe una plantilla base (`base.html`) con una barra lateral (`components/sidebar.html`) cuyas secciones se muestran según el rol del usuario, y un área de contenido principal. La estética sigue las convenciones del *skill frontend-design*: aspecto profesional e institucional, accesible (WCAG 2.2 AA) y localizado a español de México.

6.2. Componentes compartidos

Bajo `templates/solicitudes/_partials/` viven parciales reutilizados entre las vistas de solicitante y de personal: la insignia de estado (`_estado_badge.html`, que comunica el estado por color *y* texto para cumplir WCAG), la fila de solicitud, la línea de tiempo del historial, el renderizado de valores del *snapshot*, el listado de archivos y el listado de respuestas. La reutilización garantiza coherencia visual entre intake y revision.

6.3. Vistas por rol

Tabla 6.1: Superficies principales de la interfaz según el rol.

Rol	Vistas principales
Alumno / Docente	Catálogo de tipos, alta de solicitud (formulario dinámico), “Mis solicitudes” (lista filtrable), detalle con historial y documentos de respuesta.
Control Escolar / Resp. de Programa	Cola compartida por rol, detalle con acciones (atender / finalizar / cancelar), adjuntar respuesta.
Administrador	CRUD de tipos y plantillas, biblioteca de imágenes, catálogo de mentores, directorio de usuarios, tablero de reportes y exportaciones.

6.4. Accesibilidad y diseño responsivo

Las plantillas incluyen *skip-link*, un único **h1** por página, tablas con columnas de baja prioridad ocultas en pantallas estrechas, estados comunicados por color y texto, y reflujo verificado a 320 px. Los formularios marcan los campos requeridos y exponen los errores con `role="alert"`.

Capítulo 7

Seguridad

7.1. Autenticación delegada (JWT)

El sistema no gestiona credenciales: recibe un JWT de un proveedor externo (RNF-01, RT-01). El `JwtAuthenticationMiddleware` lee la cookie `stk` (o la cabecera `Authorization: Bearer`), valida la firma y la expiración con `_shared.auth.decode_jwt`, y fija `request.user` y `request.user_dto`. Ante un token inválido o expirado redirige al *login* externo. La cookie `stk` es `HttpOnly`, `SameSite=Lax`, `Secure` en producción, con `max_age` derivado de la expiración del token.

7.2. Autorización en capas

La autorización se aplica en dos niveles complementarios:

1. **Vista (grueso).** *Mixins* por rol (`CreatorRequiredMixin`, `ReviewerRequiredMixin`, `AdminRequiredMixin`) restringen el acceso a la ruta según el rol.
2. **Servicio (política de dominio).** La autorización autoritativa vive en el servicio. Por ejemplo, `LifecycleService._authorize` decide quién puede invocar cada transición; `ArchivoService` y `RespuestaService` aplican matrices de visibilidad por rol y estado; `IntakeService` re-verifica que el rol del actor pertenezca a `creator_rol` (defensa en profundidad sobre el *mixin*).

Las matrices de visibilidad de archivos y respuestas restringen la descarga al dueño de la solicitud, al personal del rol responsable y al administrador; el solicitante solo ve los documentos de respuesta cuando la solicitud está `FINALIZADA`.

7.3. CSP en la previsualización de plantillas

El editor de plantillas evalúa HTML+CSS suministrado por el administrador. Los *endpoints* de previsualización renderizan en un `iframe` con `srcdoc` y envían una cabecera `Content-Security-Policy` estricta (`default-src 'none'; style-src 'unsafe-inline'; img-src data: https: 'self'; font-src data:`) más `X-Frame-Options: SAMEORIGIN`. El `iframe` se aísla sin `allow-scripts`. Estos *endpoints* son exclusivos de administradores (`AdminRequiredMixin`); el administrador es un operador de confianza, y la superficie de plantilla queda acotada por el aislamiento del `iframe` y la restricción de rol.

7.4. Validación de archivos

Toda carga se valida por extensión y tamaño (tope global de 10 MB, RT-07) en el formulario y, de forma autoritativa, en el servicio. Las imágenes de la biblioteca se verifican adicionalmente con PIL contra los bytes reales para detectar cargas renombradas; SVG se rechaza explícitamente para evitar *scripts* embebidos. Las rutas de almacenamiento se resuelven contra `MEDIA_ROOT` rechazando intentos de salir de la raíz.

7.5. Transporte y entornos

Todo el tráfico de navegador pasa por nginx con terminación TLS (TLS 1.2+, RT-04); en producción se exige TLSv1.3, HSTS, CSP y limitación de tasa. Los secretos (`JWT_SECRET`, credenciales de SMTP, URL de SIGA) se inyectan por variables de entorno; `prod.py` falla al arrancar si falta alguno requerido, evitando despliegues mal configurados.

Capítulo 8

Tecnologías y dependencias

Tabla 8.1: Pila tecnológica del Sistema de Solicitudes.

Capa	Tecnología
Lenguaje	Python 3.12+
Framework	Django 5.x (plantillas en servidor, sin DRF)
DTOs	Pydantic v2 en cada frontera de capa
Presentación	Django Templates + Tailwind CSS v4 + Alpine.js v3 + iconos Lucide
Base de datos	PostgreSQL 16 (SQLite solo para el bucle de pruebas en proceso); ORM de Django contenido en repositorios
PDF	WeasyPrint (dependencias de sistema en la imagen)
Tareas async	Ninguna en v1 (correo síncrono con <code>try/except</code>); Celery + Redis diferidos
Almacenamiento de archivos	Sistema de archivos local (<code>media/</code>); abstracción <code>FileStorage</code> conmutable
Autenticación	Validación de JWT en <i>middleware</i> (proveedor externo)
Pruebas	<code>pytest</code> + <code>pytest-django</code> , <code>model_bakery</code> , <code>freezegun</code> , <code>responses</code> , <code>pytest-playwright</code>
Ejecución	<code>Dockerfile</code> multietapa; <code>docker-compose</code> (nginx, web, db, mailhog); <code>Makefile</code> como punto de entrada
Proxy inverso	nginx (TLS); configuraciones separadas para desarrollo y producción

8.1. Integraciones externas

Tabla 8.2: Interfaces externas del sistema.

ID	Sistema	Dirección	Datos
IE-01	Proveedor JWT	Entrada (HTTPS)	matrícula, correo, rol
IE-02	SIGA	Salida (REST/HTTPS)	nombre, programa, semestre, correo
IE-03	SMTP	Salida (SMTP/TLS)	notificaciones por evento

La consulta a SIGA es de mejor esfuerzo con *timeout* (5 s, RNF-02): ante falla se degrada usando los datos del JWT. El despacho de correo es tolerante a fallas: una caída de SMTP se registra y absorbe sin afectar la operación de negocio (RF-07).

8.2. Convenciones de construcción

El `Makefile` es el punto de entrada único; cada comando se ejecuta a través de `docker compose exec web`. Los ajustes se dividen en `config/settings/{base,dev,prod,test_postgres}.py`, con secretos por variables de entorno. El ruteo de URL es jerárquico y con espacios de nombres (proyecto → aplicación → funcionalidad), p.ej. `solicitudes:intake:create`.